

✓ Project Title: ERP System for Inventory and Sales Management – Full-Stack Development (Java + React)

👥 Client Overview:

A mid-sized distribution and trading company wants to digitize and unify its operations using a custom ERP system. The solution must allow users to manage **products, customers, sales and purchase orders, inventory updates via GRNs, and invoice generation**. The system should support **role-based access, real-time inventory tracking, and detailed financial reporting**.

🎯 Project Objective:

To build a secure, scalable ERP web application with a **Java Spring Boot REST API backend** and a **React.js frontend UI**, enabling efficient management of core business functions with database persistence, authentication, dashboards, and reporting.

🧱 Technology Stack

✓ Backend:

- Java 17+
- Spring Boot (REST API)
- Spring Data JPA (Hibernate ORM)
- Spring Security with JWT
- MySQL (or PostgreSQL)
- Swagger for API Docs
- JUnit + Mockito for testing

✓ Frontend:

- React.js (v18+)
 - Axios (API requests)
 - React Router
 - Material UI (MUI) or Bootstrap
 - React Hook Form + Yup (form validation)
 - Recharts or Chart.js for reports
 - Jest + RTL for frontend testing
-

Roles & Permissions

Role	Access Rights
Admin	Full access to all modules and reports
Sales Executive	Manage customers, create sales orders and invoices
Purchase Manager	Manage suppliers, purchase orders, GRNs
Inventory Manager	View stock, update quantities via GRN
Accountant	Manage invoices, view financial reports

ERP Modules & API Requirements

1. Product Management

Endpoints:

- GET /api/products
- POST /api/products
- PUT /api/products/{id}
- DELETE /api/products/{id}

Fields:

- Product Name
 - SKU
 - Category
 - Unit Price
 - Current Stock
 - Reorder Level
-

2. Customer & Supplier Management

Endpoints (Customer):

- GET /api/customers
- POST /api/customers

Endpoints (Supplier):

- GET /api/suppliers

- POST /api/suppliers

Fields:

- Name
 - Email
 - Phone
 - Address
 - GSTIN (optional)
-

3. Sales Order Module

Endpoints:

- GET /api/sales-orders
- POST /api/sales-orders
- PUT /api/sales-orders/{id}/status

Fields:

- Customer
 - Products & Quantity
 - Order Date
 - Status (Pending, Approved, Dispatched)
 - Total Amount (auto-calculated)
-

4. Purchase Order Module

Endpoints:

- GET /api/purchase-orders
- POST /api/purchase-orders
- PUT /api/purchase-orders/{id}/status

Fields:

- Supplier
 - Products & Quantity
 - Expected Delivery Date
 - Status (Ordered, Received)
-

5. GRN (Goods Receipt Note)

Endpoints:

- POST /api/grns: Register received goods
- GET /api/grns: View list of all GRNs

Logic:

- When a GRN is submitted, update stock of received items
-

6. Invoice Generation

Endpoints:

- GET /api/invoices
- POST /api/invoices (auto-generated from approved sales order)
- GET /api/invoices/{id}/pdf: Generate invoice as PDF

Fields:

- Customer
 - Linked Sales Order
 - Tax (GST/VAT)
 - Total Payable
 - Status (Paid, Unpaid)
-

7. Dashboard & Reporting

Endpoints:

- /api/dashboard/sales-summary
- /api/dashboard/purchase-summary
- /api/dashboard/stock-alerts

Widgets on Frontend:

- Total Sales / Purchases This Month
 - Top-Selling Products
 - Low Stock Alerts
 - Pending Invoices
-

Frontend (React.js) Screens

Navigation (Role-based):

- Sidebar navigation with modules only visible based on logged-in user role

Key Pages:

1. **Login / Register**
2. **Dashboard (Admin, Manager)**
3. **Products** (List/Add/Edit/Delete)
4. **Customers / Suppliers**
5. **Sales Order Form & Status Tracking**
6. **Purchase Orders & Status Update**
7. **GRN Form (Updates Stock)**
8. **Invoice List & PDF Viewer/Download**
9. **Reports** (charts and tables with date filters)



API Documentation

- **Swagger UI:** Auto-document all backend APIs
- Include request/response formats, sample tokens, and validation rules



Testing Strategy

Backend:

- Unit Tests: JUnit + Mockito for services and controllers
- API Tests: Postman for all endpoints

Frontend:

- Unit Tests: Jest + React Testing Library
- Functional Testing: Manual UI testing using roles



Project Timeline

Week Backend Tasks

Frontend Tasks

- | | | |
|---|----------------------------------|----------------------|
| 1 | Setup Spring Boot + Auth | Setup React + Auth |
| 2 | Product, Customer, Supplier APIs | Product, Customer UI |
| 3 | Sales Order, Purchase Order APIs | Sales/Purchase UI |

	Week Backend Tasks	Frontend Tasks
4	GRN, Invoice, Stock Update	GRN, Invoice UI
5	Dashboard Reports, Final Testing	Reports UI, Testing

Deliverables

- Spring Boot REST API backend with all modules
 - JWT-secured API access with roles
 - React.js frontend with Material UI
 - Swagger API documentation
 - Postman Collection
 - SQL or MongoDB schema
 - GitHub project with README
 - Testing reports and screenshots
 - PDF invoice generation (optional)
-

Optional Enhancements

- Email notifications (order updates, stock alerts)
 - QR code on invoices
 - Role-based dashboard graphs
 - Excel/CSV export
 - Mobile-friendly version
-

This ERP Management System project will enable you to develop a comprehensive full-stack application using **Java Spring Boot** for the backend and **React** for the frontend. It will provide practical experience in building scalable RESTful APIs for managing **products, customers, sales orders, purchase orders, GRNs, and invoices**, combined with secure user authentication using JWT and role-based access control. The project emphasizes database design with **MySQL**, performance optimization through pagination and filtering, and creating a user-friendly, responsive interface with React. Additionally, you will gain hands-on skills in API documentation with Swagger, frontend-backend integration, and testing both API endpoints and UI components, preparing you for real-world enterprise software development.